



PREMIER UNIVERSITY, CHITTAGONG
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (CSE)

PROJECT REPORT

COURSE NAME:

CSE 4001 - INTERNSHIP

PROJECT NAME:

DATABASE DESIGN AND WEB-BASED CRUD IMPLEMENTATION
FOR THE AG-LAB EVENTS & ACTIVITIES MANAGEMENT SYSTEM

SUBMITTED BY:

SHAISHOB BOIDYA

ID: 0222220005101049

SECTION: B

SEMESTER: 8th

SESSION: SPRING 2026

BATCH: 42nd

SUBMITTED TO:

IMTIAZ UDDIN RIAD

FOUNDER, TORI IT

1. Introduction

This project focuses on the design and implementation of a relational database and a full-featured, web-based CRUD (Create, Read, Update, Delete) application for the “Events & Activities” module of the AG-Lab (SUST) laboratory website. The goal of the project is to demonstrate the complete lifecycle of a database-driven web application — from relational schema design, to backend implementation using an MVC framework, to a functional and user-friendly front-end interface.

Rather than working with a static or read-only dataset, this project builds an active database system in which records can be created, viewed, searched, filtered, updated and deleted directly through a web interface, with all changes persisted in a MySQL relational database. The system manages information about laboratory events such as seminars, workshops, conferences and training sessions, including details like the event type, date, time, venue, speaker, description, status and an uploaded image.

1.1 Objectives

- Design a normalized relational database schema to store event and activity records.
- Implement full CRUD functionality (Create, Read, Update, Delete) on top of the schema using an MVC web framework.
- Enforce data integrity through server-side validation rules and controlled vocabularies (enum fields) for categorical data such as event type and status.
- Provide a searchable and filterable listing interface, backed by parameterised SQL queries generated through an ORM.
- Support image uploads for each event record, with file storage handled separately from the database.
- Demonstrate the Model-View-Controller (MVC) architectural pattern and RESTful resource routing in a real application.

The complete source code for this project — including the database migrations, models, controllers and Blade views is organised as a Laravel 11 application named “aglab-events”.

The project's GitHub repository: <https://github.com/SB-Abhi1/aglab-events>

2. Project & Database Description

The core of the system is a single primary entity, Event, which represents one lab event or activity. Every record in the events table corresponds to one event, and the table was designed to hold all the information needed to display, search and manage that event without requiring joins to other tables for the basic use case.

2.1 Main Entity: events

Field	Data Type	Description
id	BIGINT (PK, auto-increment)	Unique identifier for each event record.
title	VARCHAR(255)	Title of the event, seminar, workshop, conference or training session.
type	ENUM	Category of the event — one of: Seminar, Workshop, Event, Conference, Training.
event_date	DATE	Calendar date on which the event takes place.

event_time	TIME (nullable)	Scheduled start time of the event.
venue	VARCHAR(255, nullable)	Physical or virtual location of the event (e.g. SUST Auditorium, Online / Zoom).
speaker	VARCHAR(255, nullable)	Name of the speaker, guest or resource person.
description	TEXT	Detailed description of the event.
image	VARCHAR(255, nullable)	Relative storage path of the uploaded event image.
status	ENUM	Current status — one of: Upcoming, Ongoing, Completed, Cancelled.
created_at / updated_at	TIMESTAMP	Automatically managed audit timestamps (Laravel default).

Using ENUM columns for type and status enforces a fixed, controlled vocabulary at the database level, which prevents inconsistent or misspelled category values from being stored — functionally similar to a small lookup/reference table but without the overhead of an additional join for this scale of application.

2.2 Supporting Tables

In addition to the events table, the application ships with Laravel's default authentication-related tables (users, password_reset_tokens, sessions), plus cache and jobs tables used by the framework's caching and queue subsystems. As stated in the project scope, the Events & Activities module itself does not require authentication and exposes public CRUD access, so these tables are present in the schema for framework completeness but are not directly used by the Event feature.

2.3 Dataset Scale

The database was seeded with a small demonstration dataset using Laravel's factory and seeder classes: ten randomly generated events (via Faker) plus one realistic, manually defined seminar record, giving eleven total rows to exercise the listing, search, filter, edit and delete functionality during development and testing.

3. Tools & Technology Stack

The project was built using the Laravel PHP framework together with a MySQL relational database, following the standard Laravel project layout. The table below summarises every layer of the stack and the specific tool used for it.

Layer / Purpose	Technology Used
Backend language	PHP 8.2
Web framework	Laravel 11 (laravel/framework ^11.31)
Architecture pattern	MVC (Model – View – Controller)
ORM / Database layer	Eloquent ORM (Laravel)
Database engine	MySQL
Templating engine	Blade

Front-end styling	Bootstrap 5.3 (CDN) + Font Awesome 6 icons
Dependency management	Composer
Local development server	php artisan serve (Laragon / XAMPP environment)
Test data generation	FakerPHP + Laravel Factories & Seeders
Version control	Git

4. Development Environment Setup

The application was set up locally using Laragon as the PHP/MySQL development environment. The following sequence of steps was used to install dependencies, configure the environment, and prepare the database.

```
composer install
copy .env.example .env
php artisan key:generate
# set DB_DATABASE, DB_USERNAME, DB_PASSWORD in .env
# create the matching database in HeidiSQL / phpMyAdmin
php artisan migrate --seed
php artisan storage:link
php artisan serve
```

The php artisan migrate command reads every file in database/migrations and creates the corresponding tables in MySQL in chronological order. The --seed flag additionally runs DatabaseSeeder, which populates the events table with the demonstration records described in Section 2.3. php artisan storage:link creates a symbolic link from public/storage to storage/app/public, which is required for uploaded event images to be publicly viewable through the browser.

5. Database Design

The database schema is defined declaratively through Laravel migration files, which act as version-controlled, incremental instructions for building and modifying the database structure. This means the schema itself is part of the source code and can be recreated identically on any machine by running the migrations, rather than relying on a manually exported SQL dump.

5.1 Entity-Relationship (ER) Diagram

The conceptual model below shows the Event entity together with all of its attributes, drawn in classic Chen ER notation. The entity is represented as a rectangle at the centre, each attribute as an oval connected to it by a line, and the primary key attribute (id) is shown as a double oval with its name underlined.

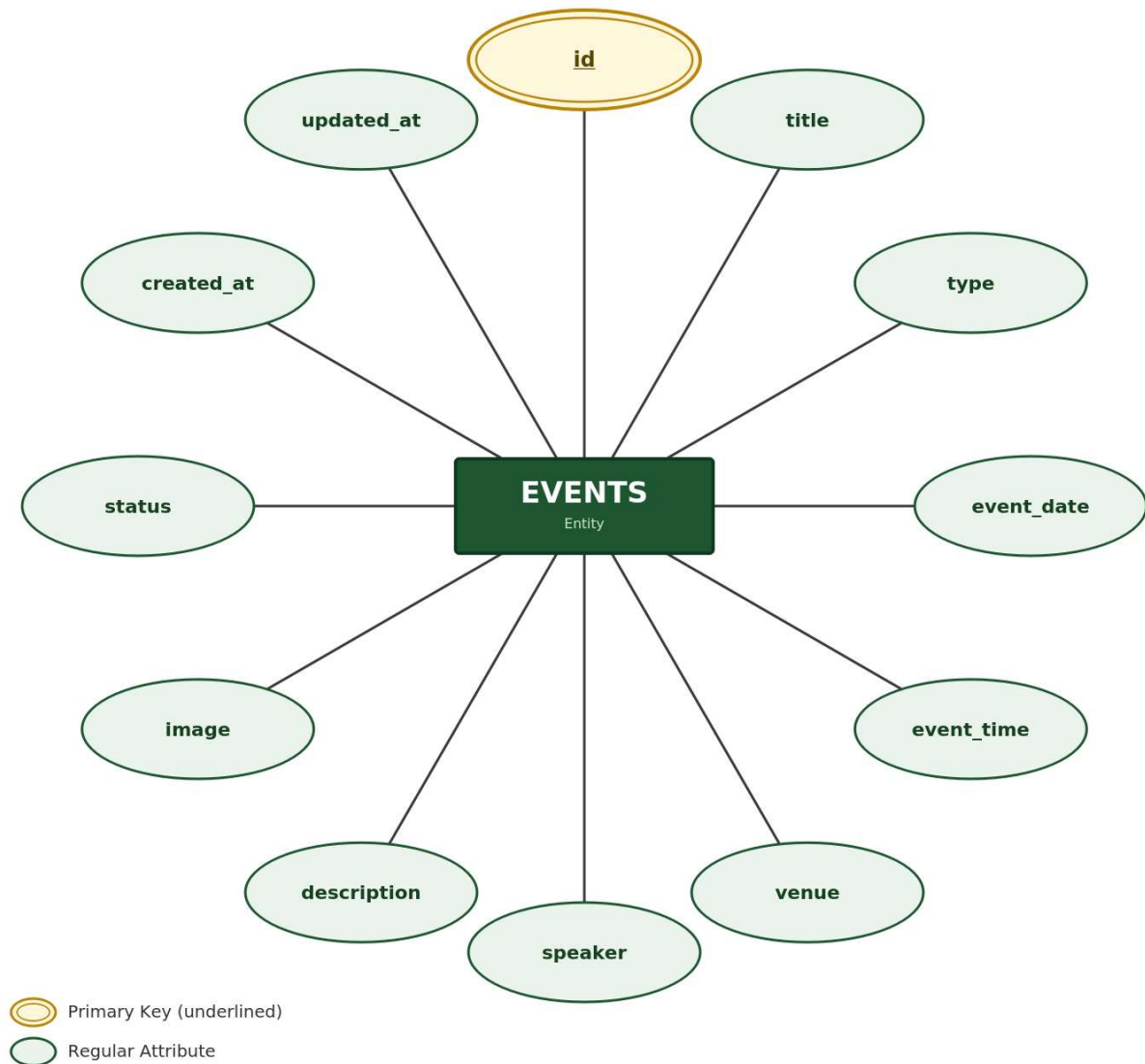


Figure 1: Entity-Relationship (ER) diagram of the events entity

5.2 Schema Diagram (Relational Table Structure)

The diagram below shows the physical relational schema that the ER model above was translated into — the actual events table as it exists in MySQL, with every column, its data type, and key/constraint notes.

Relational Schema — Table Structure

events		
Primary table — stores every lab event / activity record		
PK id	BIGINT auto-increment	Unique row identifier
title	VARCHAR(255)	Required
type	ENUM(...)	Seminar / Workshop / Event / Conference / Training
event_date	DATE	Required
event_time	TIME	Nullable
venue	VARCHAR(255)	Nullable
speaker	VARCHAR(255)	Nullable
description	TEXT	Required
image	VARCHAR(255)	Nullable — storage path
status	ENUM(...)	Upcoming / Ongoing / Completed / Cancelled
created_at	TIMESTAMP	Auto-managed
updated_at	TIMESTAMP	Auto-managed

Note: Laravel's default framework tables (users, password_reset_tokens, sessions, cache, jobs) are also present in the database but carry no foreign-key relationship to **events** and are not used by the Events & Activities feature, so they are omitted here for clarity.

Figure 2: Relational schema diagram of the events table

5.3 Migration: create_events_table

```
Schema::create('events', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->enum('type', ['Seminar', 'Workshop', 'Event',
                        'Conference', 'Training'])->default('Event');
    $table->date('event_date');
    $table->time('event_time')->nullable();
    $table->string('venue')->nullable();
    $table->string('speaker')->nullable();
    $table->text('description');
    $table->string('image')->nullable();
    $table->enum('status', ['Upcoming', 'Ongoing',
                          'Completed', 'Cancelled'])
        ->default('Upcoming');
    $table->timestamps();
});
```

5.4 Design Notes

- Primary key: id is an auto-incrementing BIGINT primary key, generated automatically by `$table->id()`.
- Controlled vocabularies: the type and status columns are ENUM types with a fixed default, which constrains the data domain directly at the database layer.
- Nullable vs required fields: event_time, venue, speaker and image are nullable because not every event has a fixed time slot, a physical venue, a named speaker, or an uploaded photo, whereas title, type, event_date, description and status are required for every record.
- Auditability: timestamps() automatically adds and maintains created_at and updated_at columns, allowing every record's creation and last-modification time to be tracked without extra application code.
- Reversibility: every migration defines a matching down() method that drops the table, so the schema change can be cleanly rolled back during development.

6. Application Architecture (MVC)

The application follows Laravel's Model-View-Controller architecture, which separates the database logic (Model), the request-handling/business logic (Controller), and the presentation layer (View / Blade templates) into distinct layers.

6.1 Model — Event.php

The Event Eloquent model maps directly to the events table. It defines a \$fillable array to protect against mass-assignment vulnerabilities, a \$casts array to automatically convert event_date into a proper PHP date object, an accessor (getImageUrlAttribute) that returns either the uploaded image URL or a placeholder image, and two reusable query scopes: scopeUpcoming() (filters and orders future events) and scopeSearch() (performs a multi-column LIKE search across title, venue and speaker).

6.2 Controller — EventController.php

EventController implements all seven standard RESTful resource actions. index() builds the query dynamically — applying the search scope and optional type/status filters from the request, then paginating the results 9 per page. store() and update() share a private validateEvent() helper so validation rules are defined exactly once and stay consistent between creating and editing a record. Image uploads are handled with Laravel's Storage facade, and the previous image file is deleted from disk whenever it is replaced during an update.

6.3 Routing — routes/web.php

A single line, `Route::resource('events', EventController::class)`, registers all seven CRUD routes automatically, following Laravel's RESTful convention:

HTTP Verb	URI	Controller Method	Purpose
GET	/events	index()	List, search and filter all events
GET	/events/create	create()	Show the “Add New Event” form
POST	/events	store()	Save a new event to the database
GET	/events/{event}	show()	Display a single event's details

GET	/events/{event}/edit	edit()	Show the “Edit Event” form
PUT / PATCH	/events/{event}	update()	Save changes to an existing event
DELETE	/events/{event}	destroy()	Remove an event (and its image file)

6.4 Views — Blade Templates

The presentation layer is built with Blade templates under `resources/views/events/`: `index.blade.php` (listing with search/filter), `create.blade.php` and `edit.blade.php` (which both reuse a single shared `_form.blade.php` partial to avoid duplicating form markup), and `show.blade.php` (single-record detail view). A common `layouts/app.blade.php` defines the shared header, navigation bar, footer and Bootstrap-based styling used across every page.

7. Features Implemented (CRUD Functionality)

This section walks through each CRUD operation as implemented in the running application, together with the corresponding screen. Paste the matching screenshot from your project into each placeholder box below.

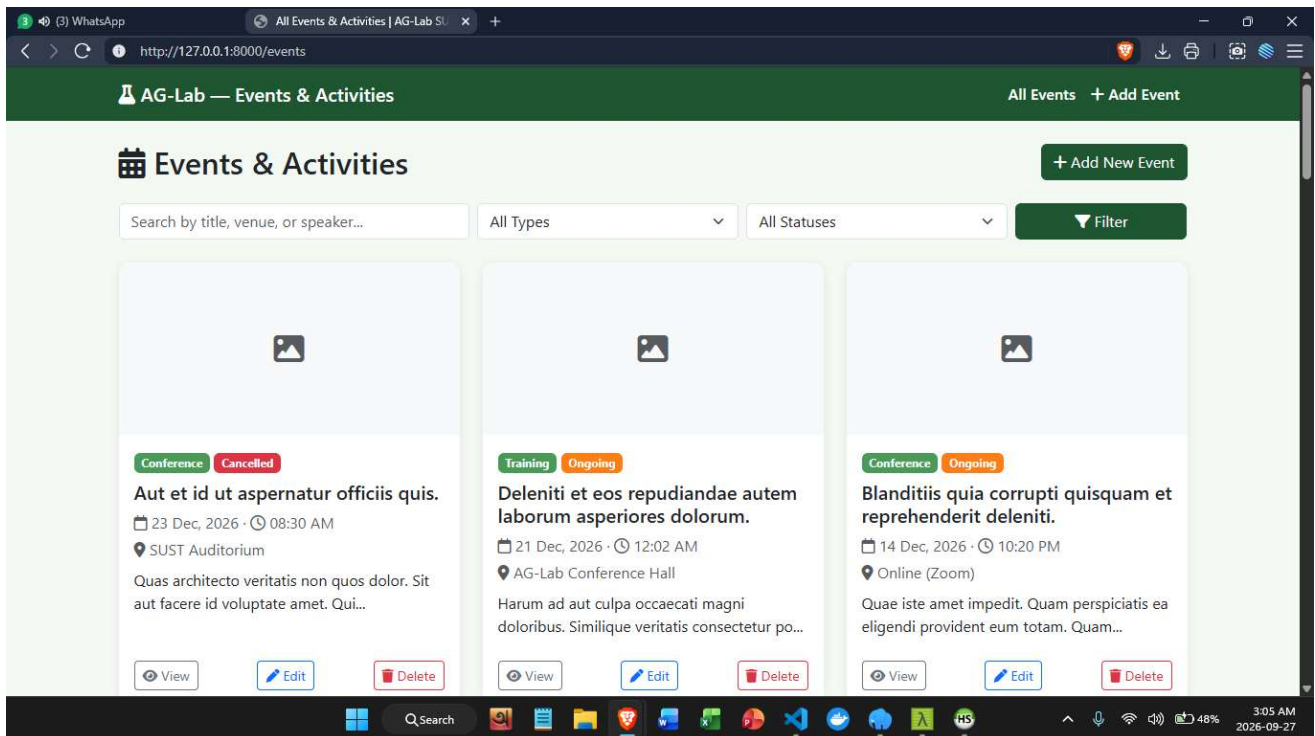
7.1 Create — Add New Event

The “Add New Event” form (`create.blade.php`) collects the title, type, date, time, status, venue, speaker, description and an optional image. On submission, the data is validated and passed to `Event::create()`, which performs a mass-assignment insert restricted to the fields listed in the model's `$fillable` array.

Add New Event / Activity form

7.2 Read — All Events Listing

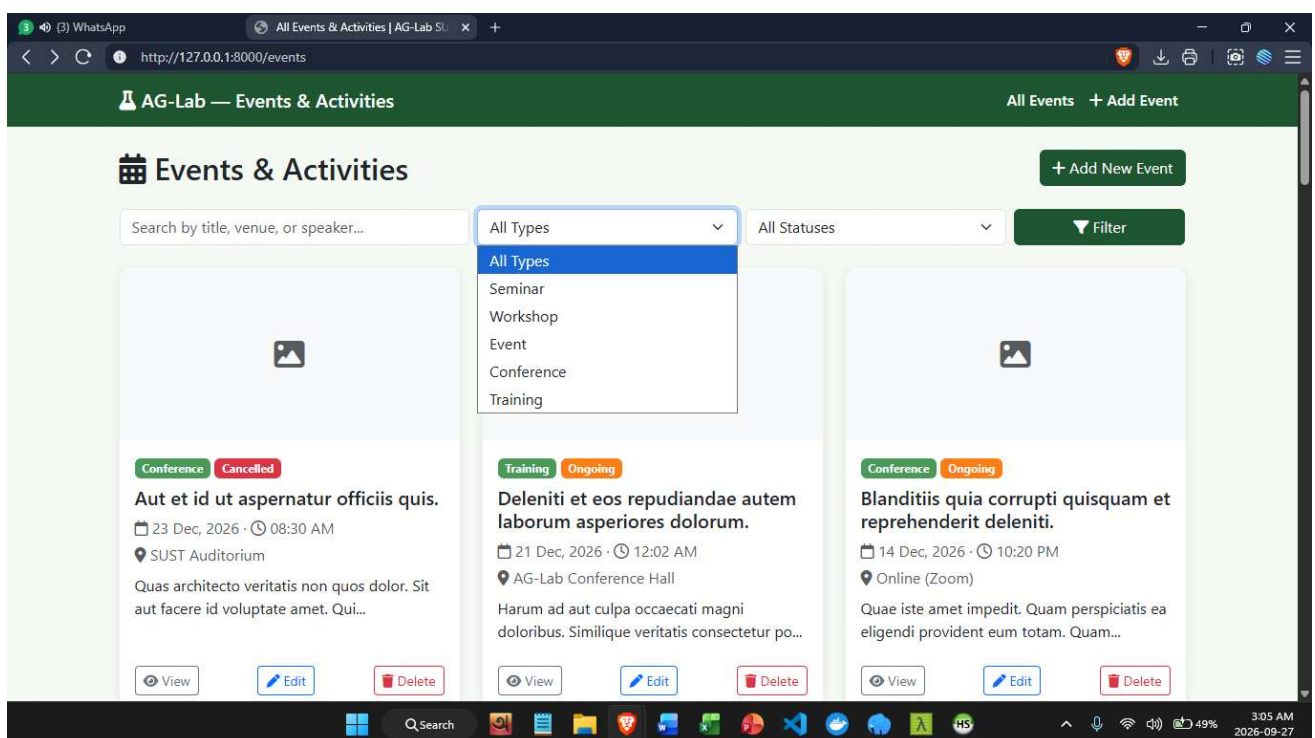
The main listing page (`index.blade.php`) displays all events as Bootstrap cards, each showing the type and status as coloured badges, the date, time, venue, a short excerpt of the description, and View / Edit / Delete action buttons. Results are paginated at 9 events per page using Eloquent's `paginate()` method.



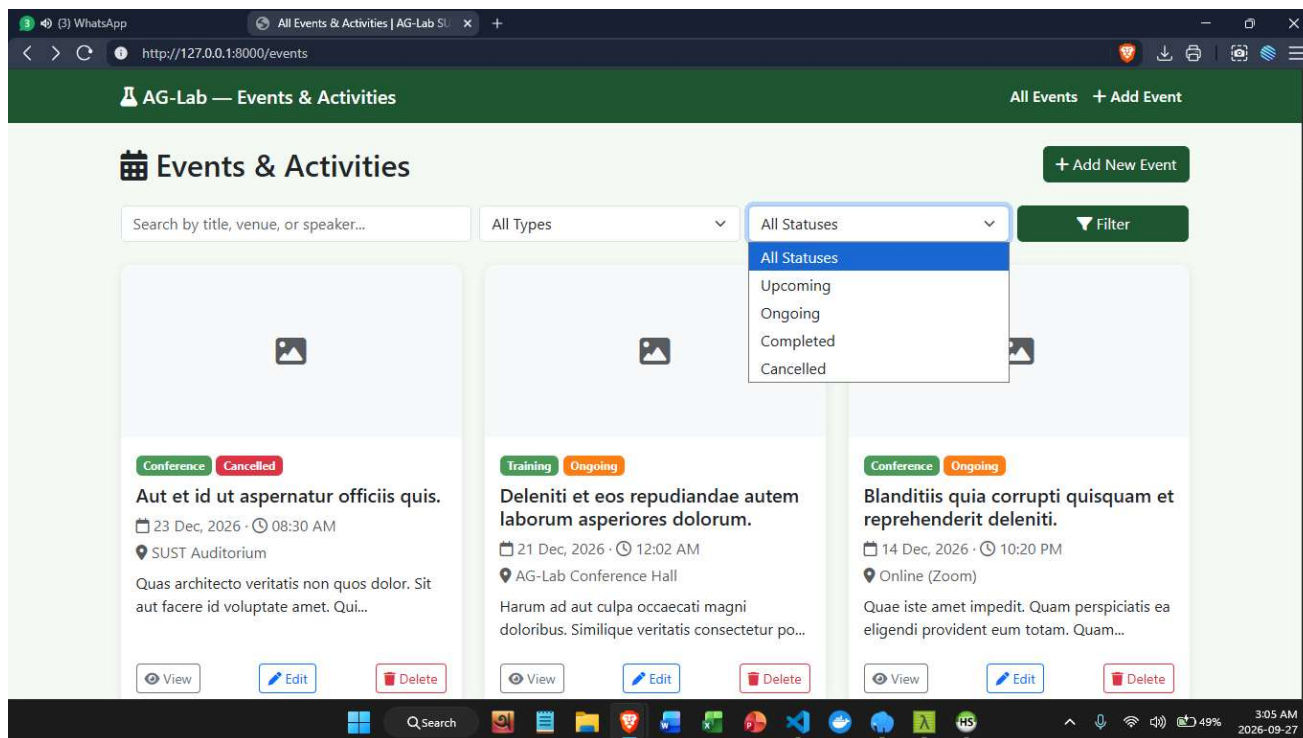
All Events & Activities — default listing view

7.3 Read — Search and Filtering

Above the listing, a search box and two dropdown filters (Type and Status) let the user narrow down the results. These are implemented with the `scopeSearch()` query scope for the free-text search box, and simple conditional `where()` clauses in the controller for the Type and Status dropdowns, combined using Eloquent's fluent, chainable query builder.



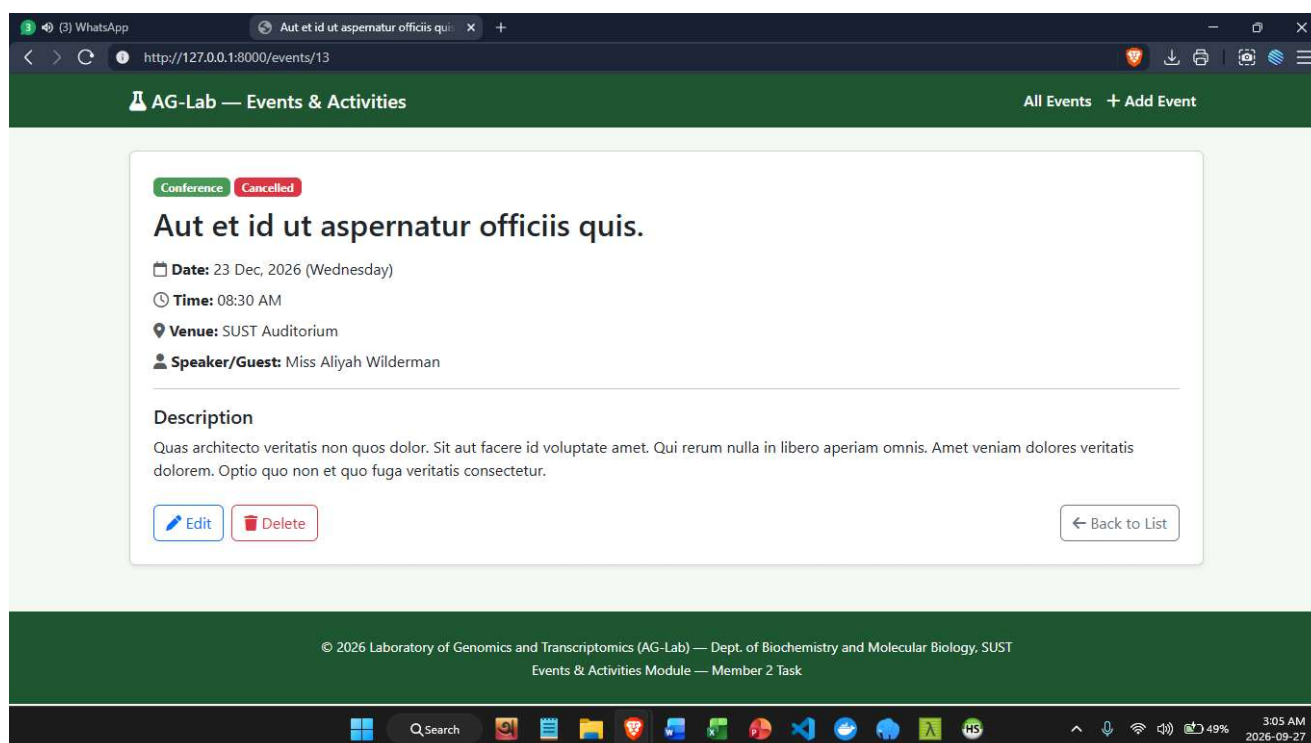
"All Types" filter dropdown expanded



"All Statuses" filter dropdown expanded

7.4 Read — Single Event Details

Clicking "View" on any event opens its dedicated detail page (show.blade.php), which displays the full description together with the date, time, venue and speaker, and provides direct Edit / Delete actions and a link back to the full list.



Single event details page

7.5 Update — Edit Event

The Edit form reuses the same `_form.blade.php` partial as the Create form, but is pre-populated with the selected event's current values. Submitting the form sends a PUT request to `update()`, which re-validates the data, optionally replaces the uploaded image (deleting the old file from storage first), and persists the changes with `$event->update()`.

Edit Event / Activity

Title *

Type *

Event Date *

Event Time

Status *

Venue

Speaker / Guest

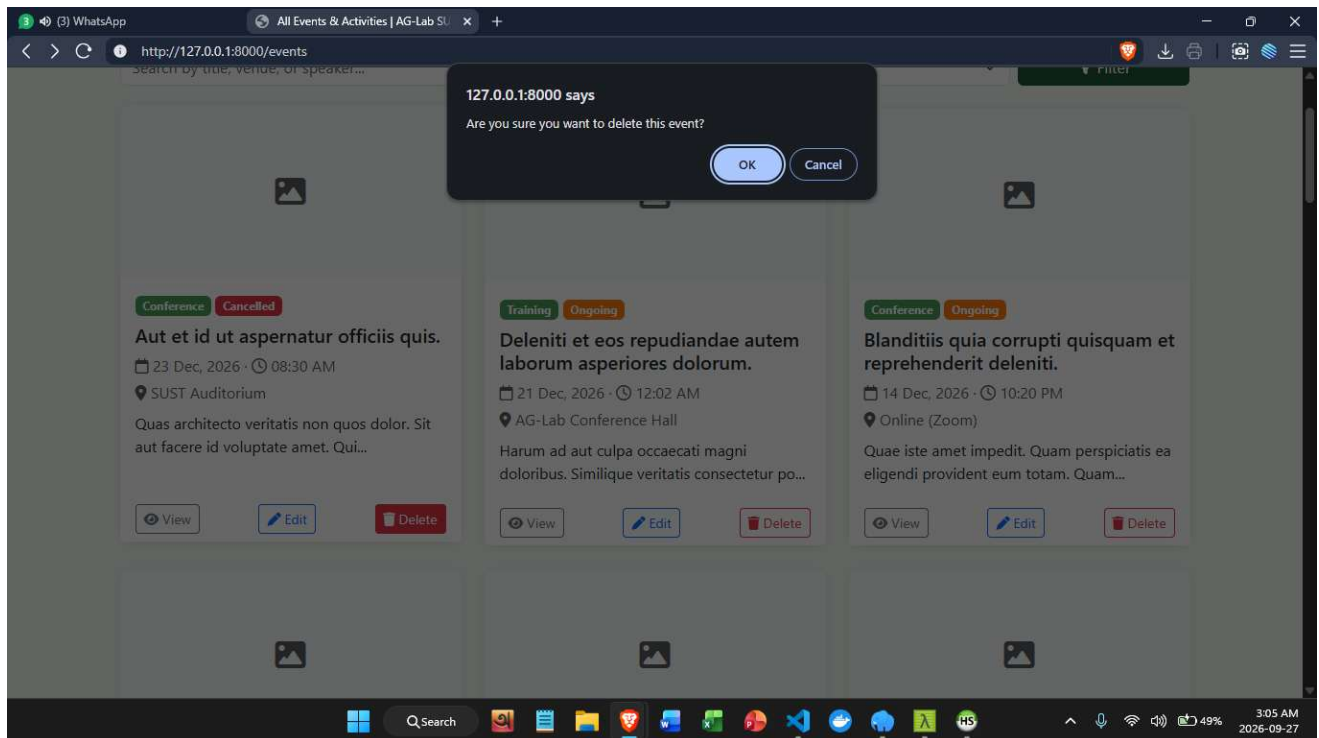
Description *

Event Image

Edit Event / Activity form, pre-filled with existing data

7.6 Delete — Remove Event

Each event card includes a Delete button that triggers a JavaScript confirm() dialog before submitting a DELETE request, preventing accidental data loss. On confirmation, destroy() removes the associated image file from storage (if one exists) and then deletes the database row.



Delete-confirmation dialog before removing an event

8. Validation & Data Integrity

Both `store()` and `update()` route through a single shared `validateEvent()` method, ensuring the same validation rules apply consistently whenever a record is created or modified:

```
'title'      => 'required|string|max:255',
'type'       => 'required|in:Seminar,Workshop,Event,Conference,Training',
'event_date' => 'required|date',
'event_time' => 'nullable|date_format:H:i',
'venue'      => 'nullable|string|max:255',
'speaker'    => 'nullable|string|max:255',
'description'=> 'required|string',
'status'     => 'required|in:Upcoming,Ongoing,Completed,Cancelled',
'image'      => 'nullable|image|mimes:jpg,jpeg,png,webp|max:2048',
```

- The `in:` rule restricts type and status to exactly the values allowed by the database ENUM, so invalid input is rejected before it ever reaches SQL.
- The image rule restricts uploads to JPG, JPEG, PNG or WEBP files no larger than 2 MB.
- Laravel's `old()` helper repopulates the form with previously entered values if validation fails, so the user does not have to retype the whole form.
- The model's `$fillable` whitelist provides a second layer of protection against mass-assignment of unintended fields.

9. Image Upload & File Storage

Uploaded event images are not stored inside the database itself; only the relative file path is stored in the image column, while the actual file is written to disk using Laravel's Storage facade (`Storage::disk('public')`). This is a standard practice that keeps the database lightweight and fast to query. The `storage:link` artisan command creates a symbolic link so files placed in `storage/app/public` are reachable through the public web root at `/storage/...`. When an event's image is replaced or the event itself is deleted, the old file is explicitly removed from disk to avoid orphaned files accumulating over time. If no image has been uploaded for an event, the model's `getImageUrlAttribute()` accessor falls back to a placeholder graphic instead.

10. Discussion

10.1 Schema Design Choices

Using ENUM columns for type and status kept the schema simple for this scope of application (a single-table CRUD module) while still enforcing a controlled vocabulary. In a larger system with many categories that change frequently, these would more naturally be split into separate lookup tables (e.g. `event_types`, `event_statuses`) referenced by foreign keys, which would allow new categories to be added without a schema migration.

10.2 Query Efficiency

Combining the search, type filter and status filter into a single chained Eloquent query, rather than fetching all rows and filtering in PHP, means MySQL performs the filtering at the database layer and only `paginate(9)` rows are ever loaded into memory per request. `withQueryString()` also preserves the active search/filter parameters across pagination links, so navigating to page 2 does not lose the user's current filter selection.

10.3 Code Reuse

Sharing a single `_form.blade.php` partial between the Create and Edit pages, and a single `validateEvent()` method between `store()` and `update()`, avoids duplicating the form markup and validation rules in two places — reducing the risk that the two forms drift out of sync as the application evolves.

10.4 Practical Challenges

As noted in the project scope, this module intentionally implements public CRUD access without authentication, since access control was outside this task's boundary. In a production deployment, the Create/Edit/Delete routes would need to sit behind authentication and authorization middleware so that only authorised lab members could modify event records, while the public listing and detail pages remained open to all visitors.

11. Conclusion

This project implemented a complete, database-backed CRUD web application for managing lab events and activities, covering the full pipeline from relational schema design through to a working, styled front end. The events table was designed with appropriate data types, nullable fields and ENUM-based controlled vocabularies; the Laravel MVC architecture cleanly separated the Eloquent model, the resourceful controller, and the Blade views; and the resulting interface supports creating, listing, searching, filtering, viewing, editing and deleting event records, with image upload handled through file storage rather than the database itself.

Overall, the project provided hands-on experience with relational database design, Eloquent ORM query building, RESTful resource routing, server-side validation, and file storage management within a real-world-style Laravel application.